

Claude Code KV- 캐시를 살려둘 가치가 있는가 ?

개념부터 차근차근 — 구독 과금 환경에서 cache-keepalive 플러그인 실측 A/B 검증

결론 : 이 환경에서 플러그인은 비용을 줄이지 못하고 구독 쿼터만 소모한다 — 캐시 유효시간 (TTL) 이 1 시간이라 7 분 정도 쉬는 것으로는 캐시가 스스로 살아있고 , 킵얼라이브 핑은 절약 없이 5 시간 요청 쿼터만 깎는다 . 끄는 것을 권고한다 .

이 보고서를 읽는 순서

01

배경 개념 : 프롬프트 캐시란 무엇인가

02

배경 개념 : 구독 과금 vs API 과금

03

이 실험이 알아내려던 것 (A/B 설계)

04

실측 결과 : 캐시 상태 · 히트율 · TTL 분해

05

판정과 권고

개념 ① : 프롬프트 캐시 — LLM 이 매 턴 대화 전체를 다시 읽는 비용을 줄이는 장치

01 프리픽스 재전송

매 턴 LLM 은 지금까지의 대화 전체 (프리픽스) 를 다시 입력으로 읽어야 함

02 cache write (비쌈)

처음 읽을 때 캐시에 저장 — 입력 토큰의 1.25x(5 분)~2.0x(1 시간) 가중 비용

03 cache read (싼)

이미 저장된 프리픽스를 재사용 — 입력 토큰의 0.1x, 즉 10 분의 1

04 TTL 만료 → 콜드

유효시간 (TTL) 이 지나면 캐시가 사라짐 → 다시 비싼 write 발생

캐시가 '따뜻하다' vs '콜드' 다 — 같은 대화를 이어가도 비용이 10 배 이상 갈린다

따뜻한 캐시 (cache read)

- TTL 이 아직 안 지나 프리픽스가 살아있음
- 재사용 비용 = 입력 토큰 x 0.1 (싼 read)
- 유희 (쉬는 구간) 가 TTL 보다 짧으면 자동으로 유지됨



콜드 캐시 (cache write 재발생)

- TTL 이 지나 프리픽스가 사라짐
- 처음부터 다시 저장 = x1.25~x2.0 (비싼 write)
- 유희가 TTL 보다 길 때만 발생

개념 ② : 구독 과금 vs API 과금 — ' 무엇으로 비용이 났는가 ' 가 다르다

구독 과금 (Claude Pro)

- 5 시간 롤링 윈도우의 ' 요청 수 ' 로 청구
- 토큰을 많이 써도 한 요청은 한 요청
- 캐시로 토큰을 아껴도 요청 쿼터는 그대로 났음

API 과금 (API 키)

- 실제 사용한 ' 토큰 비용 ' 으로 청구
- 토큰을 아끼면 곧바로 돈이 절약됨
- 캐시 read(0.1x) 가 직접적인 비용 절감

그래서 핵심 질문은 하나 : 이 환경에서 캐시는 절약을 만드는가 ?

0.1x vs 1.25x

cache read 대 cache write 의 가중 비용 배수 — 플러그인은 비싼 write 를싼 read 로 바꿔 절약하려 함

단 , 구독 과금에선 절약된 토큰이 ' 돈 ' 이 아니라 ' 요청 쿼터 ' 와 무관하다 . 핑 한 번은 요청 한 장을 그대로 소모한다 .

개념 ③ : 이 실험이 알아내려던 세 가지

이 환경의 실제 캐시 유효시간 (TTL) 은 몇 분인가

- 플러그인은 '5 분 TTL' 을 전제로 핑을 보냄
- 전사 로그에서 ephemeral_5m vs ephemeral_1h write 를 분해해 확인

7 분 쉬면 캐시는 따뜻하게 살아남는가 , 콜드가 되는가

- 실제 벽시계 유티 직후 첫 턴의 캐시 read/write 를 측정
- 유티 경계 (gap-boundary) 에서 캐시 생존 여부를 직접 관찰

플러그인이 캐시 히트율 (CHR) 을 실제로 끌어올리는가

- $CHR = \text{캐시 read} / (\text{read} + \text{write})$, 1 에 가까울수록 캐시 효율 좋음
- ON 암과 OFF 암의 CHR 을 나란히 비교

요약하면 : TTL 이 정말 5 분인지 , 7 분 유티로 콜드가 되는지 , 플러그인이 효과가 있는지를 실측으로 확인한다 .

동일 베이스라인 , 실제 벽시계 유희로 구성한 5 개 라이브 암

워크로드 2 종 × 플러그인 ON/OFF + 70 분 TTL 탐침 암 1 종 (각 암 n=1)

A	B	C	D	E
wlA-off - 워크로드 A · TS API JSDoc - 플러그인 OFF 7 분 유희	wlA-on - 워크로드 A · TS API JSDoc - 플러그인 ON 7 분 유희	wlB-off - 워크로드 B · Python 오류경로 - 플러그인 OFF 7 분 유희	wlB-on - 워크로드 B · Python 오류경로 - 플러그인 ON 7 분 유희	ttl-long - TTL 한계 탐침 - 플러그인 OFF 70 분 단일 유희
각 암 = 일회용 워크트리 · n=1				

이 숫자들은 어떻게 모았나 — 토큰 사용량은 Anthropic 서버가 응답마다 직접 돌려준다

01 서버가 usage 반환

매 응답에
cache_read·cache_write·ephemeral_5m/
1h·output 토큰 수가 포함됨 (추정이 아닌
실측치)

02 전사 JSONL 기록

Claude Code 가 세션 대화를
~/.claude/projects/.../< 세션 >.jsonl 에
메시지 단위로 저장

03 파서로 토큰별 추출

harness/parse-usage.py 가 어시스턴트
턴마다 usage 를 읽고 real/keepalive 로
분류

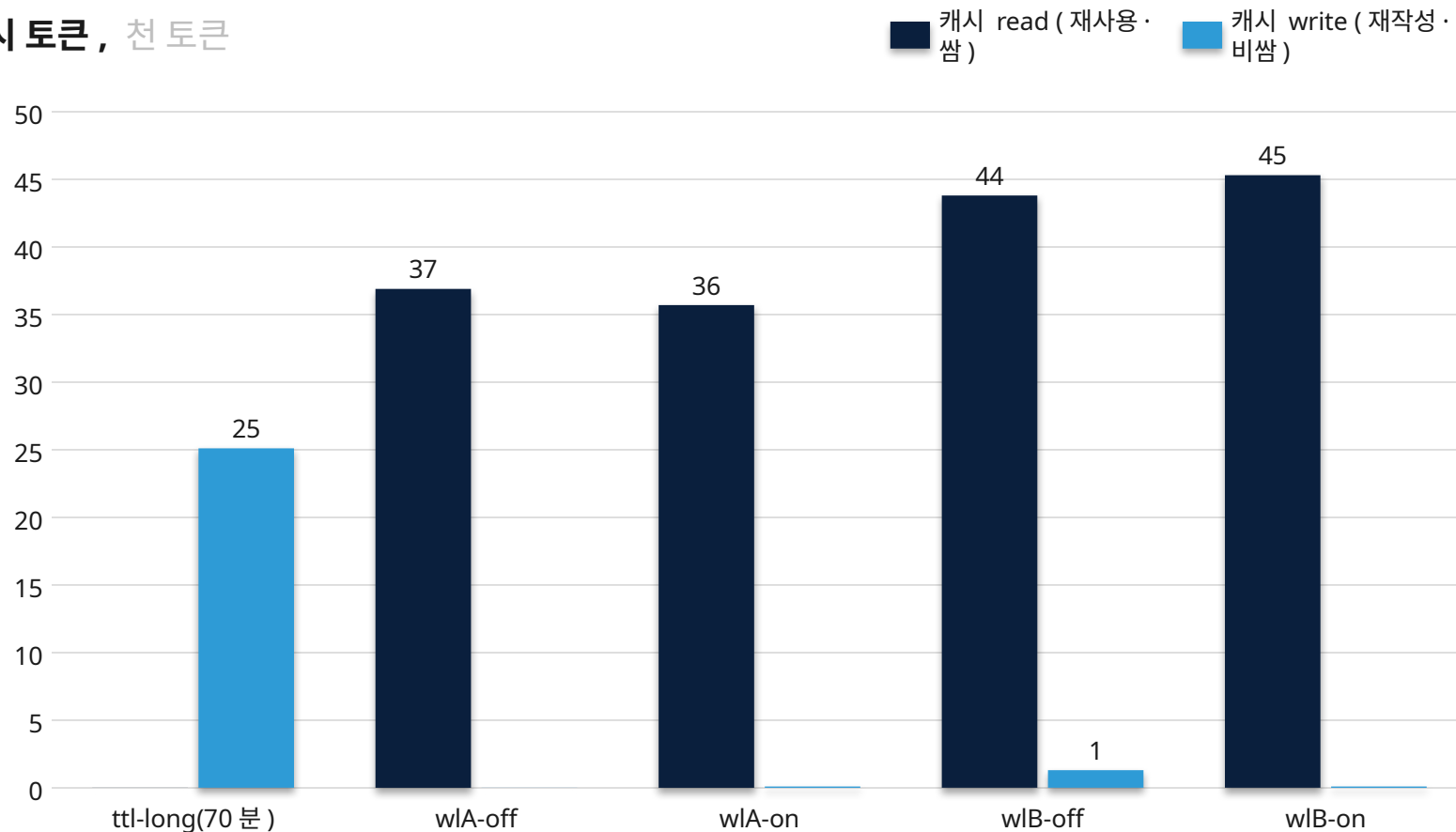
04 암별 집계 · 교차검증

CHR·CWT 로 합산하고 jq 로 원본 전사 합계와
대조해 일치 확인

7 분 유희는 캐시를 따뜻하게 유지 , 70 분 유희만 콜드로 전환

유희 (쉬는 구간) 직후 첫 실행에서 읽힌 캐시 read 토큰과 새로 쓴 write 토큰

캐시 토큰 , 천 토큰



핵심 시사점

- ttl-long 만 read 0 · write 25,134 — 캐시가 사라져 완전 재작성
- 7 분 4 개 암은 read 가 우세 → 캐시가 그대로 살아있었음
- 즉 실제 TTL 은 7 분보다 길고 70 분보다 짧음 (문서상 1 시간과 일치)

약 150 톤 동안 '5 분 캐시'에 쓴 토큰은 단 한 개도 없었다

100%

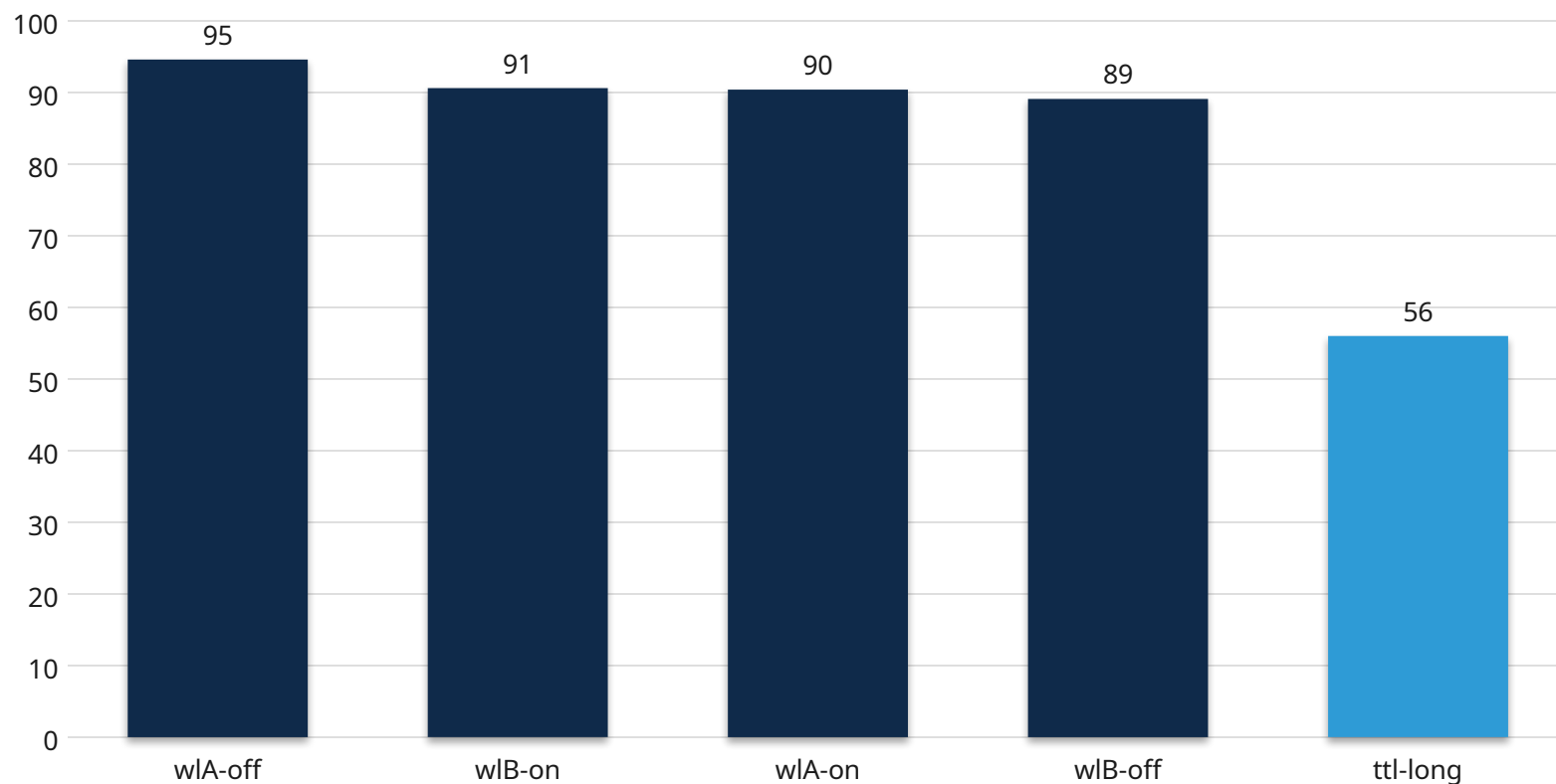
ephemeral_1h(1 시간 TTL)를 사용한 write 토큰 비중 — 총 466,136 개 전부

ephemeral_5m(5 분 TTL) write = 0 개 . 플러그인이 격파하려는 '5 분 TTL'은 이 환경에 아예 존재하지 않는다 .

따뜻한 암은 캐시 히트율 89~95%, 콜드 70 분 암만 56% 로 붕괴

캐시 히트율 (CHR) = 캐시 read / (read + write) · 1 에 가까울수록 캐시가 잘 재사용됨

캐시 히트율 , %



핵심 시사점

- 플러그인 ON(90.4%) 과 OFF(94.6%) 가 사실상 동일
- ON·OFF 차이는 캐시 효과가 아니라 워크로드 노이즈
- 즉 플러그인은 캐시 히트율을 끌어올리지 못함
- 콜드 암 (ttl-long) 만 유일하게 56% 로 급락

플러그인은 작동은 한다 — 다만 값싼 read 한 번을 보낼 뿐, 콜드가 없어 무의미

wlA-on · t27 킵얼라이브 턴

- 캐시 read 35,691 토큰 (이미 따뜻한 캐시를 읽음)
- 캐시 write 25 토큰 (거의 0)
- 출력 22 토큰 — 0.1x 값싼 read 한 번

wlB-on · t28 킵얼라이브 턴

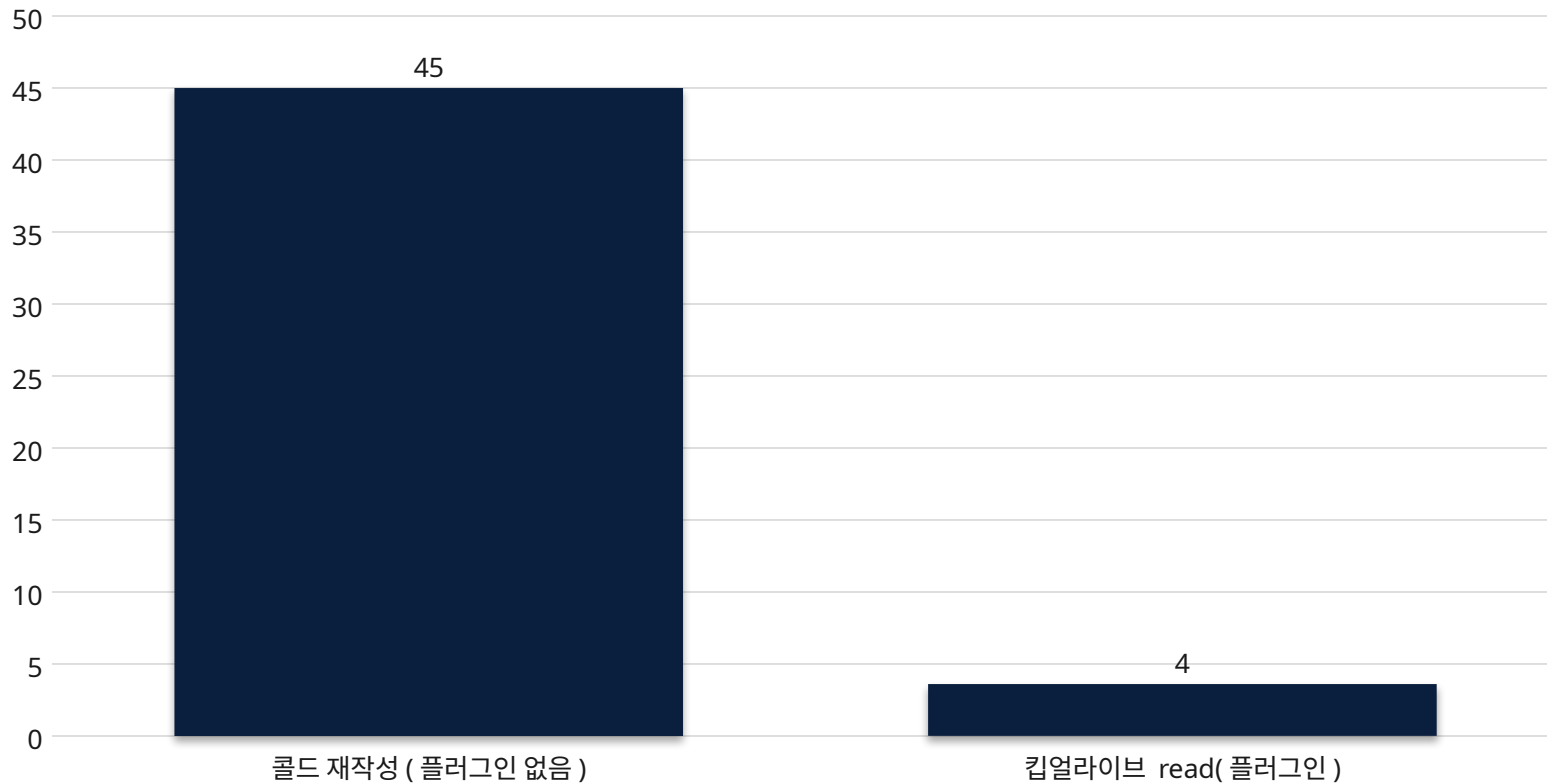
- 캐시 read 44,355 토큰 (이미 따뜻한 캐시를 읽음)
- 캐시 write 909 토큰 (거의 0)
- 출력 54 토큰 — 0.1x 값싼 read 한 번

만약 5 분 TTL·API 과금이었다면 유희당 약 41,000 가중 토큰을 아꼈을 것

약 36k 프리픽스 기준 — 유희 한 번당 발생하는 가중 토큰 (반사실 시나리오)

가중 토큰, 천 토큰

■ 가중 토큰



반사실 시나리오

- 콜드 재작성 = $36k \times 1.25 \approx 45,000$ 가중 토큰
- 킵얼라이브 read = $36k \times 0.10 \approx 3,600$ 가중 토큰
- 유희당 절약 약 41,400 — 단, 요청은 +1 회 추가
- Aider·Cline 류 환경에선 유효하나, 여기선 레버가 연결돼 있지 않음

토큰 비용 (API) 은 거의 안 움직이고 , 요청 비용 (구독) 은 오르기만 한다

환경 · 유효 조건별 플러그인 판정 (Harvey-ball: 채울수록 유리)

Criteria	플러그인 없음	플러그인 사용
이 환경 (1h TTL)·7 분 유효		
가상 5 분 TTL·7 분 유효		
이 환경 ·1 시간 초과 유효		

★ Recommended

여기선 플러그인을 끈 채로 — 과금·TTL 이 바뀔 때만 재검토

실행 : 이 구독 설치에서 **cache-keepalive** 비활성 유지

- 현재 1 시간 TTL 에서는 순수 오버헤드 (절약 0)
- 7 분 유틸은 그 자체로 캐시를 따뜻하게 유지함

재검토 트리거 : API- 키 과금 전환 + 전사에 **ephemeral_5m write** 관측

- 두 조건이 동시에 성립할 때만 플러그인이 유효
- TTL 은 계약이 아닌 환경 변수 (1h→5m 로 바뀐 전례 2026/3)

조치 : 조건 충족 시 **harness/run-all.py** 재실행으로 재측정

- 반사실 분석 (슬라이드 14) 이 실제 API 비용 절감을 예측

현 환경 권고 : 플러그인 OFF 유지 . 과금 방식 또는 TTL 이 변할 때에만 재검토 .

방법론 견고성과 한계

견고한 근거

- ✓ 유희 경계 캐시 상태는 턴 수에 무관하게 robust
- ✓ 캐시 히트율 (CHR) 도 턴 수 영향 없이 robust
- ✓ 70 분 단일 유희 암이 TTL 을 7~70 분으로 고정

한계 및 주의

- ✗ 암당 $n=1$ (반복 측정 없음)
- ✗ Claude 비결정성으로 암 간 총량 교란 (45 턴 vs 35 턴)
- ✗ TTL 을 분 단위로 정확히 특정하지는 못함